

A simple example of multiple imputation using the mice package

Brice Ozenne

September 29, 2026

This document gathers code from the documentation of the mice package. See <https://stefvanbuuren.name/mice/>.

Load packages

```
library(lava)
library(mice)
library(data.table)
library(ggplot2)
```

1 Simulate data

Just to have an example to work with. Generative model:

```
mSim <- lvm(Y~group+season+bmi+gender+age)
categorical(mSim, labels = c("winter", "summer")) <- ~season
categorical(mSim, labels = c("SAD", "HC")) <- ~group
categorical(mSim, labels = c("Male", "Female")) <- ~gender
distribution(mSim, ~bmi) <- lava::gaussian.lvm(mean = 22, sd = 3)
distribution(mSim, ~age) <- lava::uniform.lvm(20, 80)
```

Sampling

```
n <- 1e2
set.seed(10)
dt.data <- as.data.table(sim(mSim, n))
```

Add missing values

```
dt.data[1:10, "age" := NA]
dt.data[11:20, c("age", "bmi", "Y") := NA]
```

2 Working with mice

2.1 Step 1: Inspect the missing data pattern

Check the number of missing values in the dataset:

```
colSums(is.na(dt.data))
```

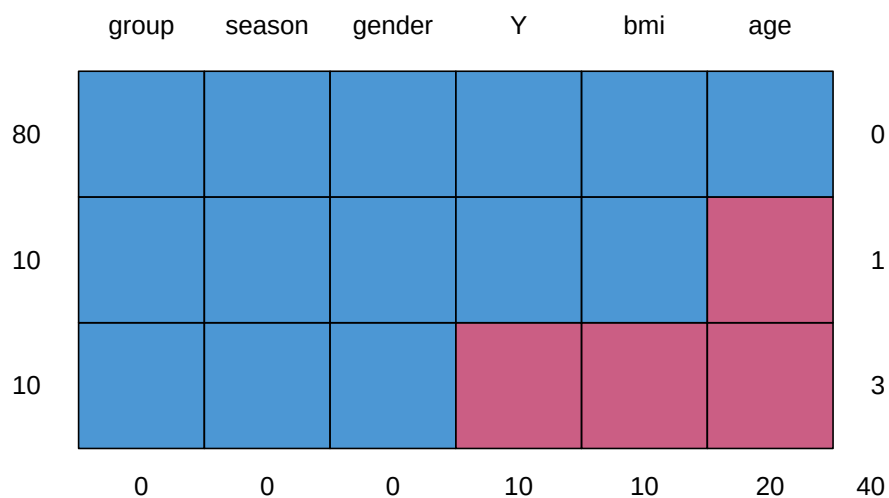
```
Y group season    bmi gender    age
10    0      0     10    0     20
```

Missing data patterns:

```
md.data <- md.pattern(dt.data)
md.data
```

```
group season gender Y bmi age
80      1      1      1 1  1  1  0
10      1      1      1 1  1  0  1
10      1      1      1 0  0  0  3
      0      0      0 10 10 20 40
```

- the left column counts the number of observations having a specific missing data pattern.
- the right column display the number of missing data in each pattern
- the bottom row counts the number of missing value relative to each variable (and the total number of missing values).



2.2 Step 2: Reflect upon the missing data and correlation patterns

It is often useful to consider the missing data patterns next to the association between variables. Ideally the association pattern should be based on prior knowledge as the empirical one (i.e. based on the observed data) may be difficult to estimate reliably in presence of missing data. To keep things concrete, let's use a rough approximation (pearson correlation also for categorical variables) of the correlation structure:

```
dtNum.data <- as.data.frame(lapply(dt.data,as.numeric))
cor.data <- cor(dtNum.data, use = "pairwise")
cor.data[setdiff(colnames(md.data),""),
          setdiff(colnames(md.data), "")]
```

	group	season	gender	Y	bmi	age
group	1.00000000	-0.08242959	-0.04222003	0.11147368	-0.09679738	0.09304925
season	-0.08242959	1.00000000	-0.09090909	0.09219243	0.06508765	0.07620663
gender	-0.04222003	-0.09090909	1.00000000	-0.12831302	0.04329831	-0.17662044
Y	0.11147368	0.09219243	-0.12831302	1.00000000	0.32446590	0.98452932
bmi	-0.09679738	0.06508765	0.04329831	0.32446590	1.00000000	0.16195300
age	0.09304925	0.07620663	-0.17662044	0.98452932	0.16195300	1.00000000

Intuition: A complete case analysis where Y is regressed against **group**, adjusting for **season**, **gender**, **bmi**, **age** would discard information from all the 20 subjects with missing values.

- Complete case would discard valuable information for the 10 subjects where Y is known but **age**. Indeed, **age** being very correlated with Y, those subjects are not really missing data. Multiple imputation can avoid discarding such valuable information, leading to superior statistical power and possibly less bias.
- the 10 subjects missing **bmi**, **age**, and Y carry little information as **group**, **season**, and **gender** do not seem to be particularly useful to guess the missing values. In that case multiple imputation would essentially impute the group mean for **bmi**, **age**, and Y leading to similar results as complete case analysis.

Multiple imputation (MI) vs. complete case (CC) (van Buuren, 2018):

"Suppose that the complete-data model is a regression with outcome Y and predictors X . If the missing data occur in Y only, complete-case analysis and multiple imputation are equivalent [...] Multiple imputation gains an advantage over complete-case analysis if additional predictors for Y are available that are not part of X . The efficiency of complete-case analysis declines if X contains missing values. [...] there are two special cases where listwise deletion outperforms multiple imputation [...]

- the probability to be missing does not depend on Y
- the complete data model is logistic regression [...] the probability to be missing depends only on Y and not on X "

2.3 Step 3: Define imputation model

```
all.variables <- setdiff(colnames(md.data), "")
n.variables <- length(all.variables)
Mlink <- matrix(0, n.variables, n.variables,
               dimnames = list(all.variables, all.variables))
Mlink["Y", which(colnames(Mlink) != "Y")] <- 1
Mlink["age", which(colnames(Mlink) != "age")] <- 1
Mlink["bmi", which(colnames(Mlink) != "bmi")] <- 1
Mlink
```

	group	season	gender	Y	bmi	age
group	0	0	0 0	0	0	0
season	0	0	0 0	0	0	0
gender	0	0	0 0	0	0	0
Y	1	1	1 0	1	1	1
bmi	1	1	1 1	0	1	1
age	1	1	1 1	1	1	0

A value of 1 means that the column variable is used as a predictor for the target block (in the rows).

2.4 Step 4: Generate imputed datasets

Generate imputed values

```
n.imputed <- 3 ## number of imputed datasets
dt.mice <- mice(dt.data,
               m=n.imputed,
               maxit = 100, # number of iterations to obtain the imputed
               dataset
               predictorMatrix = Mlink,
               method = 'pmm', # Predictive mean matching, only ok for
               continuous variables, it is possible to set constrains for positive
               variables
               seed = 500, printFlag = FALSE)
summary(dt.mice)
```

Class: mids

Number of multiple imputations: 3

Imputation methods:

group	season	gender	Y	bmi	age
""	""	""	"pmm"	"pmm"	"pmm"

PredictorMatrix:

	group	season	gender	Y	bmi	age
group	0	0	0	0	0	0
season	0	0	0	0	0	0
gender	0	0	0	0	0	0
Y	1	1	1	0	1	1
bmi	1	1	1	1	0	1
age	1	1	1	1	1	0

2.4.1 Interacting with the mice object

Missingness indicator:

```
table(cci(dt.mice))
```

FALSE	TRUE
20	80

Complete case dataset:

```
str(cc(dt.mice))
```

```
'data.frame':      80 obs. of  6 variables:
 $ Y      : num  48.7 91.6 75.3 51.8 92 ...
 $ group  : Factor w/ 2 levels "SAD","HC": 1 2 1 1 2 1 2 1 1 1 ...
 $ season: Factor w/ 2 levels "winter","summer": 1 1 2 1 1 2 2 2 2 2 ...
 $ bmi    : num   23.4 25.7 20.2 24.3 23.8 ...
 $ gender: Factor w/ 2 levels "Male","Female": 2 2 2 1 1 2 1 1 2 2 ...
 $ age    : num   24.8 66.1 53.7 29.7 68.4 ...
```

Extract observations with missing values:

```
str(ic(dt.mice))
```

```
'data.frame':      20 obs. of  6 variables:
 $ Y      : num   92.2 91 63.1 76.2 75.2 ...
 $ group  : Factor w/ 2 levels "SAD","HC": 1 2 1 2 1 2 2 1 2 2 ...
 $ season: Factor w/ 2 levels "winter","summer": 2 2 2 2 1 2 2 1 1 1 ...
 $ bmi    : num   21.8 25.5 28.6 23.2 19.8 ...
 $ gender: Factor w/ 2 levels "Male","Female": 2 2 1 2 2 2 1 2 1 1 ...
 $ age    : num   NA NA NA NA NA NA NA NA NA NA ...
```

Dataset after multiple imputation:

```
str(complete(dt.mice, action = 1)) ## first imputed dataset
```

```
'data.frame':     100 obs. of  6 variables:
 $ Y      : num   92.2 91 63.1 76.2 75.2 ...
 $ group  : Factor w/ 2 levels "SAD","HC": 1 2 1 2 1 2 2 1 2 2 ...
 $ season: Factor w/ 2 levels "winter","summer": 2 2 2 2 1 2 2 1 1 1 ...
 $ bmi    : num   21.8 25.5 28.6 23.2 19.8 ...
 $ gender: Factor w/ 2 levels "Male","Female": 2 2 1 2 2 2 1 2 1 1 ...
 $ age    : num   72.2 61.6 36.7 51.7 56.3 ...
```

```
str(complete(dt.mice, action = 2)) ## second imputed dataset
```

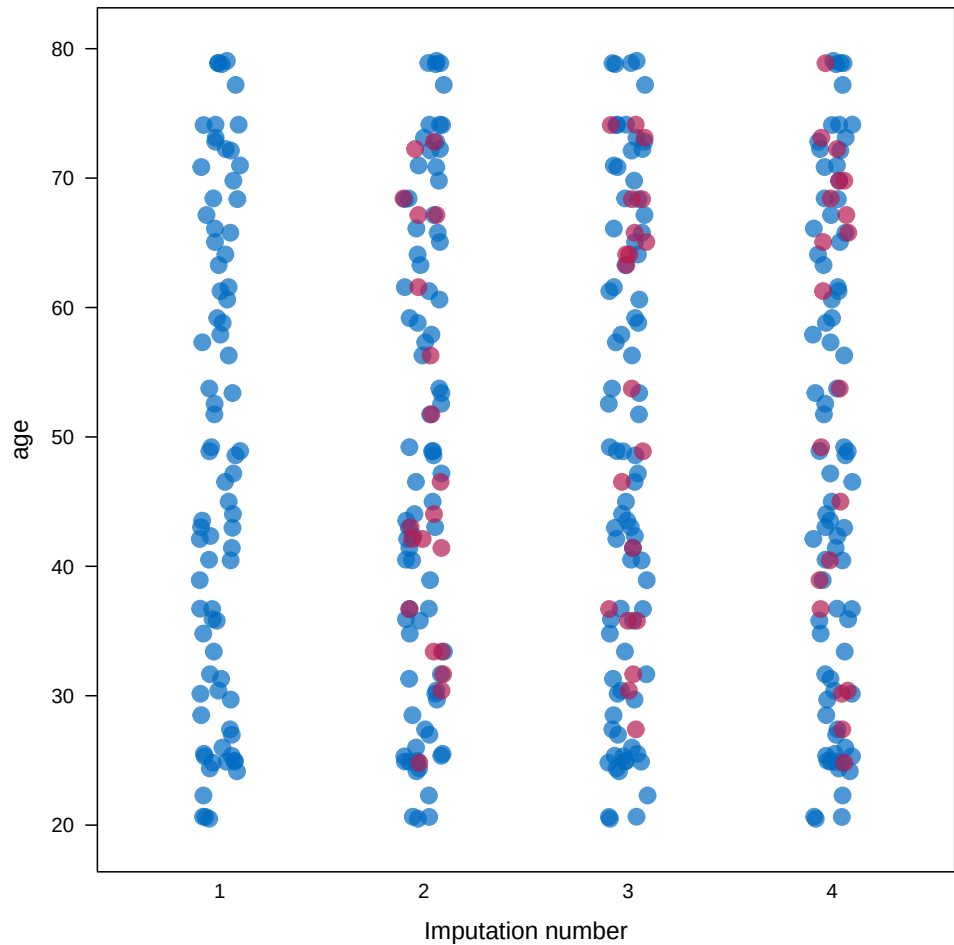
```
'data.frame':     100 obs. of  6 variables:
 $ Y      : num   92.2 91 63.1 76.2 75.2 ...
 $ group  : Factor w/ 2 levels "SAD","HC": 1 2 1 2 1 2 2 1 2 2 ...
 $ season: Factor w/ 2 levels "winter","summer": 2 2 2 2 1 2 2 1 1 1 ...
 $ bmi    : num   21.8 25.5 28.6 23.2 19.8 ...
 $ gender: Factor w/ 2 levels "Male","Female": 2 2 1 2 2 2 1 2 1 1 ...
 $ age    : num   68.4 64.1 30.4 48.9 53.7 ...
```

2.5 Step 5: Check the imputed datasets

2.5.1 Build-in plot to visualize missing values

Dotplot of the original data (blue) vs. imputed data (pink) to check that the imputed values are plausible (e.g. no negative age):

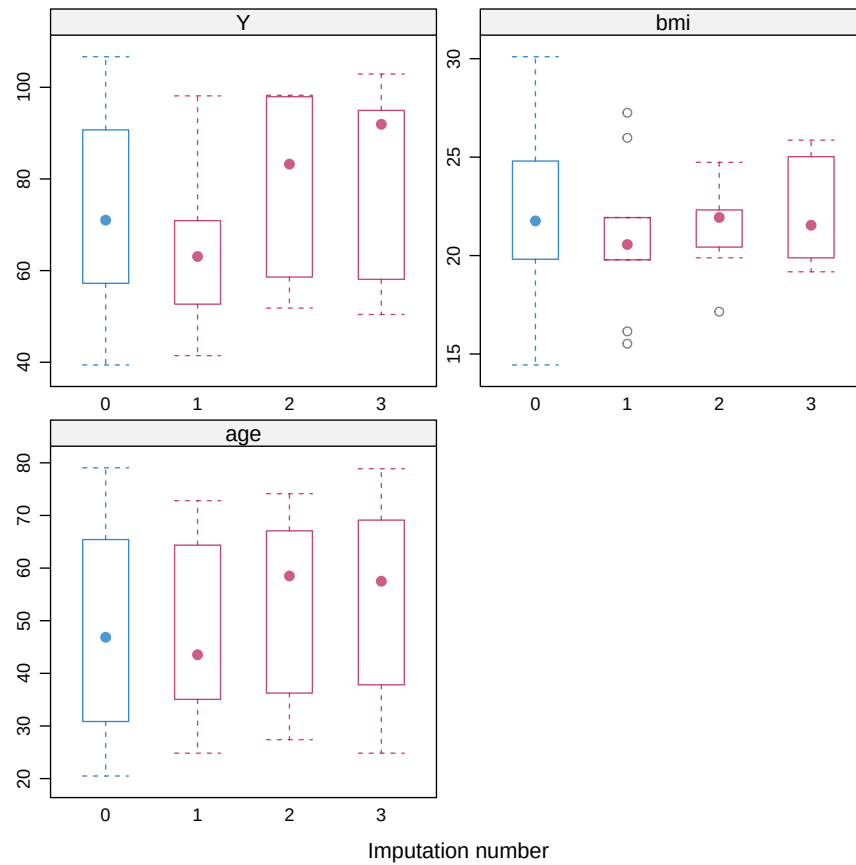
```
stripplot(dt.mice, age~.imp, pch=20, cex=2)
```



- Points above x=1 represent the original data
- Points above x=2 represent the first imputed dataset
- Points above x=3 represent the second imputed dataset
- ...

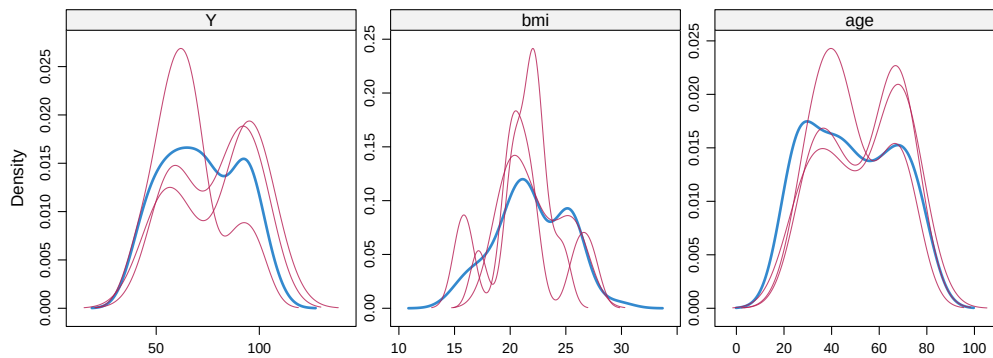
Boxplot of the original data (blue) vs. imputed data (pink) for a more compact visualization:

```
stripplot(dt.mice, age~.imp, pch=20, cex=2)
```



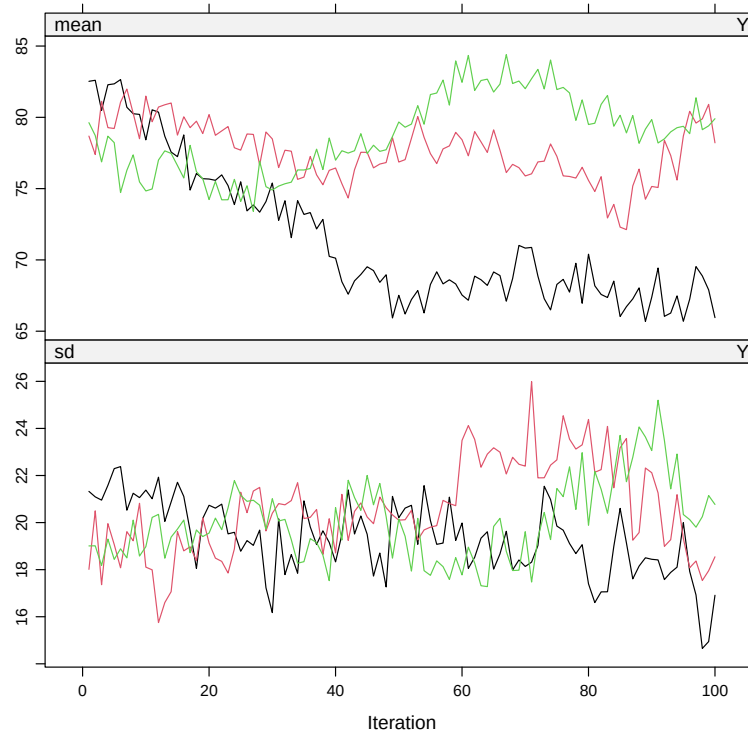
Density plot of the original data (blue) vs. imputed data (pink) to compare more in detail the distributions:

```
densityplot(dt.mice, layout = c(3,1))
```



2.5.2 Convergence of the imputation algorithm

```
plot(dt.mice, layout = c(1,2))
```



Should look more like the bottom plot than the top plot where the chains are not mixing very well. Note that here at least 50 iteration are needed to reach a steady state.

2.5.3 Make your own plot

Imputed values can be directly extracted from the object:

<code>head(dt.mice\$imp\$age)</code>	<code>tail(dt.mice\$imp\$age)</code>
--------------------------------------	--------------------------------------

	1	2	3		1	2	3
1	67.14724	20.49185	51.74262	15	31.30446	22.29032	69.79847
2	33.40963	24.89930	51.74262	16	35.79714	22.29032	24.91940
3	31.30446	43.02380	48.88883	17	35.79714	24.89930	48.88883
4	31.30446	22.29032	48.88883	18	33.40963	22.29032	24.91940
5	35.79714	31.66111	69.79847	19	31.30446	43.02380	48.88883
6	31.30446	31.66111	69.79847	20	31.30446	22.29032	48.88883

The columns correspond to the 3 different imputed datasets and the rows to 20 imputed values per dataset.

The imputed datasets can be extracted using:

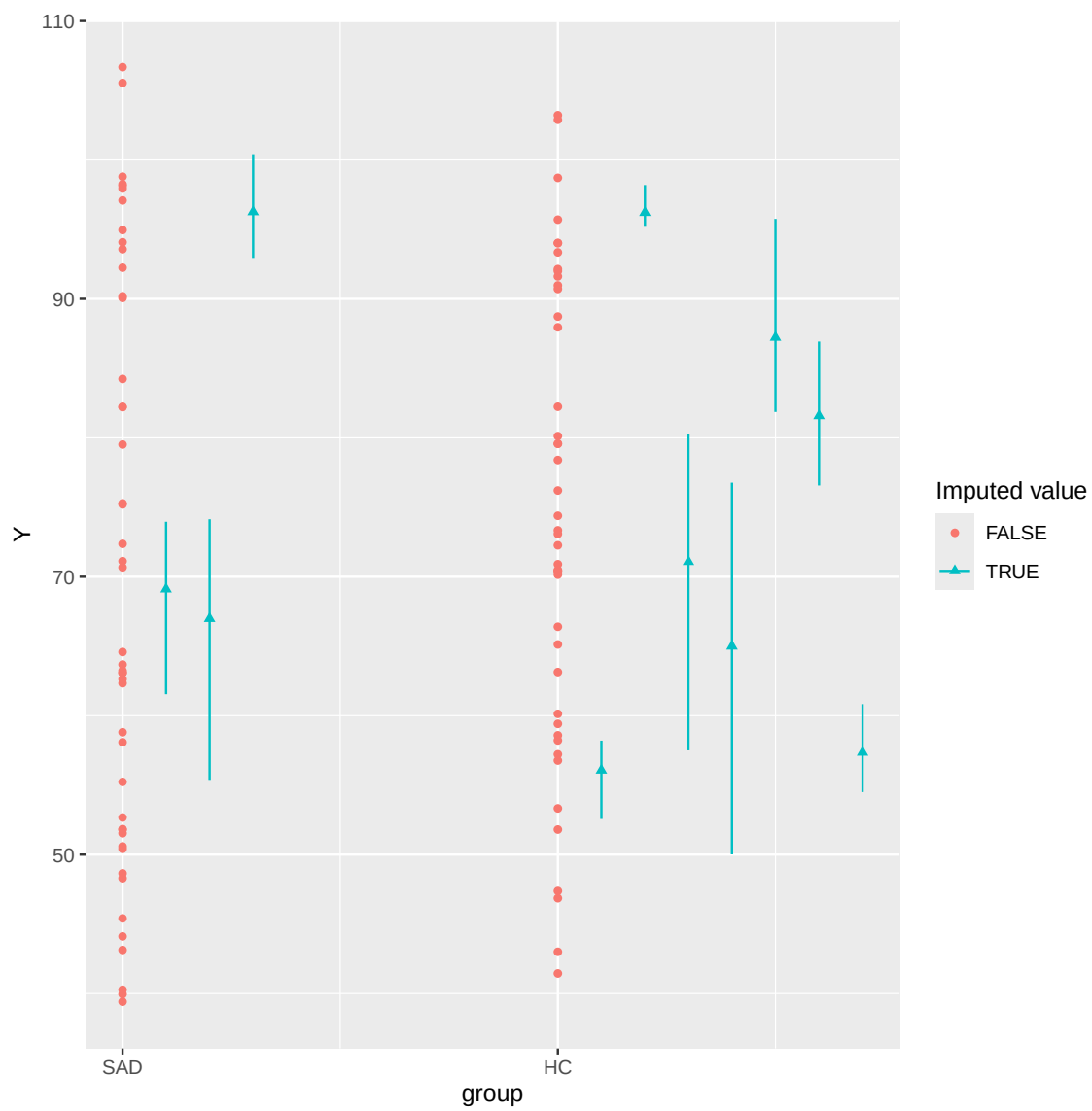
```
dfMICE.data <- complete(dt.mice, action = "long")
str(dfMICE.data)
```

```
'data.frame':      300 obs. of  8 variables:
 $ Y      : num  92.2 91 63.1 76.2 75.2 ...
 $ group  : Factor w/ 2 levels "SAD","HC": 1 2 1 2 1 2 2 1 2 2 ...
 $ season : Factor w/ 2 levels "winter","summer": 2 2 2 2 1 2 2 1 1 1 ...
 $ bmi    : num  21.8 25.5 28.6 23.2 19.8 ...
 $ gender : Factor w/ 2 levels "Male","Female": 2 2 1 2 2 2 1 2 1 1 ...
 $ age    : num  72.2 61.6 36.7 51.7 56.3 ...
 $ .imp   : int  1 1 1 1 1 1 1 1 1 1 ...
 $ .id    : int  1 2 3 4 5 6 7 8 9 10 ...
```

One can then display the observed and imputed values, possibly with an interval to reflect the sampling variation of the imputed value (here 1st and 3rd quantiles)

```
library(LMMstar)
dfMICE.mean <- summarize(Y ~ .id + group, data = dfMICE.data)
dfMICE.mean$impute <- dfMICE.mean$sd>0
dfMICE.mean$group.num <- as.numeric(dfMICE.mean$group)
dfMICE.mean$Nimpute <- unlist(tapply(dfMICE.mean$impute, dfMICE.mean$group,
  cumsum))*(dfMICE.mean$impute)

ggMICE.point <- ggplot(dfMICE.mean,
  aes(x = group.num+0.1*Nimpute, y = mean))
ggMICE.point <- ggMICE.point + geom_point(aes(color = impute,
  shape = impute))
ggMICE.point <- ggMICE.point + geom_segment(data = dfMICE.mean[dfMICE.mean
  $impute,],
  aes(color = impute, y = q1,
  yend = q3))
ggMICE.point <- ggMICE.point + scale_x_continuous(
  breaks = 1:2,
  labels = levels(dfMICE.mean$group)
)
ggMICE.point <- ggMICE.point + labs(x = "group", y = "Y",
  color = "Imputed value",
  shape = "Imputed value")
ggMICE.point
```



2.6 Step 6: Fit the statical model on each imputed dataset

```
e.mice <- with(data = dt.mice,  
              lm(Y~group+season+bmi+gender+age)  
              )  
e.mice
```

call :

```
with.mids(data = dt.mice, expr = lm(Y ~ group + season + bmi +  
  gender + age))
```

call1 :

```
mice(data = dt.data, m = n.imputed, method = "pmm", predictorMatrix = Mlink,  
      maxit = 100, printFlag = FALSE, seed = 500)
```

nmis :

```
[1] 10  0  0 10  0 20
```

analyses :

```
[[1]]
```

Call:

```
lm(formula = Y ~ group + season + bmi + gender + age)
```

Coefficients:

(Intercept)	groupHC	seasonsummer	bmi	genderFemale	age
-0.02614	0.95181	1.13110	0.99033	0.70594	1.00116

```
[[2]]
```

Call:

```
lm(formula = Y ~ group + season + bmi + gender + age)
```

Coefficients:

(Intercept)	groupHC	seasonsummer	bmi	genderFemale	age
0.05021	0.59320	0.87012	1.01371	0.59669	1.00034

```
[[3]]
```

Call:

```
lm(formula = Y ~ group + season + bmi + gender + age)
```

Coefficients:

(Intercept)	groupHC	seasonsummer	bmi	genderFemale	age
0.5906	0.7486	1.0359	0.9588	1.0171	1.0077

We can check that using `with` is equivalent to run the linear regression on the imputed dataset:

```
dt.tempo <- copy(dt.data)
dt.tempo[is.na(bmi), bmi := dt.mice$imp$bmi[,1]]
dt.tempo[is.na(age), age := dt.mice$imp$age[,1]]
dt.tempo[is.na(Y), Y := dt.mice$imp$Y[,1]]
lm(Y ~ group + season + bmi + gender + age, data = dt.tempo)
```

Call:

```
lm(formula = Y ~ group + season + bmi + gender + age, data = dt.tempo)
```

Coefficients:

(Intercept)	groupHC	seasonsummer	bmi	genderFemale	age
-0.02614	0.95181	1.13110	0.99033	0.70594	1.00116

2.7 Step 7: Pool the results over the imputed datasets

```
ePool.mice <- pool(e.mice)
summary(ePool.mice)
```

	term	estimate	std.error	statistic	df	p.value
1	(Intercept)	0.2048883	0.970756912	0.2110604	38.881759	8.339425e-01
2	groupHC	0.7645464	0.319142478	2.3956273	9.220665	3.956182e-02
3	seasonsummer	1.0123677	0.286505516	3.5335015	18.085773	2.359316e-03
4	bmi	0.9876296	0.049550963	19.9315925	9.684670	3.472961e-09
5	genderFemale	0.7732397	0.353001211	2.1904733	6.590121	6.701387e-02
6	age	1.0030637	0.008348779	120.1449616	15.554399	1.645574e-24

The (pooled) estimate is the average of the estimates relative to each imputed dataset:

```
Q.coef <- colMeans(do.call(rbind,lapply(e.mice$analyses, coef)))
Q.coef
```

(Intercept)	groupHC	seasonsummer	bmi	genderFemale	age
0.2048883	0.7645464	1.0123677	0.9876296	0.7732397	1.0030637

The variance is a bit more complex and involves:

- the within-imputation variance (depends on the sample size)

```
covW <- Reduce("+",lapply(e.mice$analyses, vcov))/n.imputed
print(covW, digits = 5)
```

	(Intercept)	groupHC	seasonsummer	bmi	genderFemale	age
(Intercept)	0.7916558	-0.04307788	-0.02717933	-0.02939118	-0.03847724	-0.00154551
groupHC	-0.0430779	0.05873233	0.00490851	0.00083376	0.00140149	-0.00017523
seasonsummer	-0.0271793	0.00490851	0.05882881	-0.00042661	0.00549819	-0.00002796
bmi	-0.0293912	0.00083376	-0.00042661	0.00144421	-0.00074300	-0.00004101
genderFemale	-0.0384772	0.00140149	0.00549819	-0.00074300	0.06116567	0.00035449
age	-0.0015455	-0.00017523	-0.00002796	-0.00004101	0.00035449	0.00004799

- the between-imputation variance (depends on the amount of missing data)

```
ls.diffCoef <- lapply(e.mice$analyses, function(iI){coef(iI)-Q.coef})
covB <- Reduce("+",lapply(ls.diffCoef,tcrossprod))/(n.imputed-1)
print(covB, digits = 5)
```

	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]
[1,]	0.1130349	-1.1447e-02	0.00182317	-0.00788221	0.06845745	1.3253e-03
[2,]	-0.0114468	3.2340e-02	0.02311661	-0.00175256	0.00688497	1.8197e-05
[3,]	0.0018232	2.3117e-02	0.01744245	-0.00203333	0.01143001	1.3527e-04
[4,]	-0.0078822	-1.7526e-03	-0.00203333	0.00075832	-0.00590408	-1.0488e-04
[5,]	0.0684574	6.8850e-03	0.01143001	-0.00590408	0.04758314	8.7018e-04
[6,]	0.0013253	1.8197e-05	0.00013527	-0.00010488	0.00087018	1.6284e-05

- the simulation error

```
covE <- covB/n.imputed
print(covE, digits = 5)
```

	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]
[1,]	0.03767828	-3.8156e-03	0.00060772	-2.6274e-03	0.02281915	4.4177e-04
[2,]	-0.00381559	1.0780e-02	0.00770554	-5.8419e-04	0.00229499	6.0656e-06
[3,]	0.00060772	7.7055e-03	0.00581415	-6.7778e-04	0.00381000	4.5090e-05
[4,]	-0.00262740	-5.8419e-04	-0.00067778	2.5277e-04	-0.00196803	-3.4962e-05
[5,]	0.02281915	2.2950e-03	0.00381000	-1.9680e-03	0.01586105	2.9006e-04
[6,]	0.00044177	6.0656e-06	0.00004509	-3.4962e-05	0.00029006	5.4279e-06

The total variance is:

```
covT <- covW + covB + covE
```

leading to the standard errors:

```
sqrt(diag(covT))
```

(Intercept)	groupHC	seasonsummer	bmi	genderFemale	age
0.970756912	0.319142478	0.286505516	0.049550963	0.353001211	0.008348779

There is also a function to extract the R-squared:

```
pool.r.squared(e.mice)
```

	est	lo 95	hi 95	fmi
R ²	0.9961692	0.9942275	0.9974586	0.06584253

```
vec.rsquared <- sapply(e.mice$analyses, function(iImp){
  summary(iImp)$r.squared
})
tanh(mean(atanh(vec.rsquared)))
```

```
[1] 0.9961692
```

3 Special case: stratified imputation and variable transformation

Mice can be adapted to use a group-specific imputation model¹:

```
dtMale.data <- dt.data[dt.data$gender=="Male",]
dtFemale.data <- dt.data[dt.data$gender=="Female",]

all.variables <- c("Y","group","season","bmi","age","gender")
n.variables <- length(all.variables)
Mlink <- matrix(0, n.variables, n.variables,
               dimnames = list(all.variables,all.variables))
Mlink["bmi",c("group","season","age")] <- 1
Mlink

## stratified imputation
dtMale.mice <- mice(dtMale.data,
                  m=10,
                  maxit = 50,
                  predictorMatrix = Mlink,
                  method = c("", "", "", "norm", "", ""),
                  seed = 500, printFlag = FALSE)
dtFemale.mice <- mice(dtFemale.data,
                    m=10,
                    maxit = 50,
                    predictorMatrix = Mlink,
                    method = c("", "", "", "norm", "", ""),
                    seed = 500, printFlag = FALSE)
dt.mice <- rbind(dtMale.mice, dtFemale.mice)

## sanity check
test <- complete(dt.mice)$bmi
GS <- c(complete(dtMale.mice)$bmi,
        complete(dtFemale.mice)$bmi)
range(test-GS)
```

	Y	group	season	bmi	age	gender
Y	0	0	0	0	0	0
group	0	0	0	0	0	0
season	0	0	0	0	0	0
bmi	0	1	1	0	1	0
age	0	0	0	0	0	0
gender	0	0	0	0	0	0

¹[//github.com/amices/mice/issues/34](https://github.com/amices/mice/issues/34)

```
Advarselsbesked:
Number of logged events: 1
Advarselsbesked:
Number of logged events: 1
[1] 0 0
```

and to define new variables post-imputation¹:

```
dtLong.mice <- complete(dt.mice, action='long', include=TRUE)
dtLong.mice$newvar <- dtLong.mice$Y^2
dt2.mice <- as.mids(dtLong.mice)

## sanity check
range(complete(dt2.mice)$newvar - complete(dt.mice)$Y^2)

## run model
elm.mice <- with(dt2.mice, lm(newvar ~ group+season+bmi+age+gender))
summary(pool(elm.mice))
```

```
[1] 0 0
```

	term	estimate	std.error	statistic	df	p.value
1	(Intercept)	-4389.00375	386.635258	-11.3517939	23.29985	5.655387e-11
2	groupHC	-53.95254	82.370728	-0.6549965	83.82218	5.142635e-01
3	seasonsummer	172.60520	84.141766	2.0513617	75.16262	4.371633e-02
4	bmi	115.09828	16.840419	6.8346443	19.81157	1.275398e-06
5	age	148.31890	2.280725	65.0314676	84.14586	0.000000e+00
6	genderFemale	203.09995	84.971650	2.3902084	69.70597	1.954547e-02

4 Special case: imputation using a specific law and no covariate

Mice can be adapted in order, for instance, to sample from a uniform distribution or a truncated normal distribution. First define a function able to generate data like:

```
mice.impute.SI_unif <- function(y, ry, ...){ ## uniform xglaw
  n.NA <- sum(ry==FALSE)
  sample <- runif(n.NA, min = 0, max = 1)
  return(cbind(sample))
}
```

or

```
mice.impute.SI_tnorm <- function(y, ry, ...){ ## truncated normal law
  require(truncnorm)
  n.NA <- sum(ry==FALSE)
  sample <- rtruncnorm(n.NA, a = 0, b = 1, mean = 1, sd = 0.1)
  return(cbind(sample))
}
```

Then prepare the matrix indicating which variable should be used during the imputation:

```
impute.var <- c("bmi", "group")
Mlink2 <- matrix(0,
                 nrow = length(impute.var),
                 ncol = length(impute.var),
                 dimnames = list(impute.var, impute.var))
Mlink2["bmi", "group"] <- 1
Mlink2
```

```
      bmi group
bmi    0    1
group  0    0
```

Then run mice as usual except that the method should correspond to one of the previous functions:

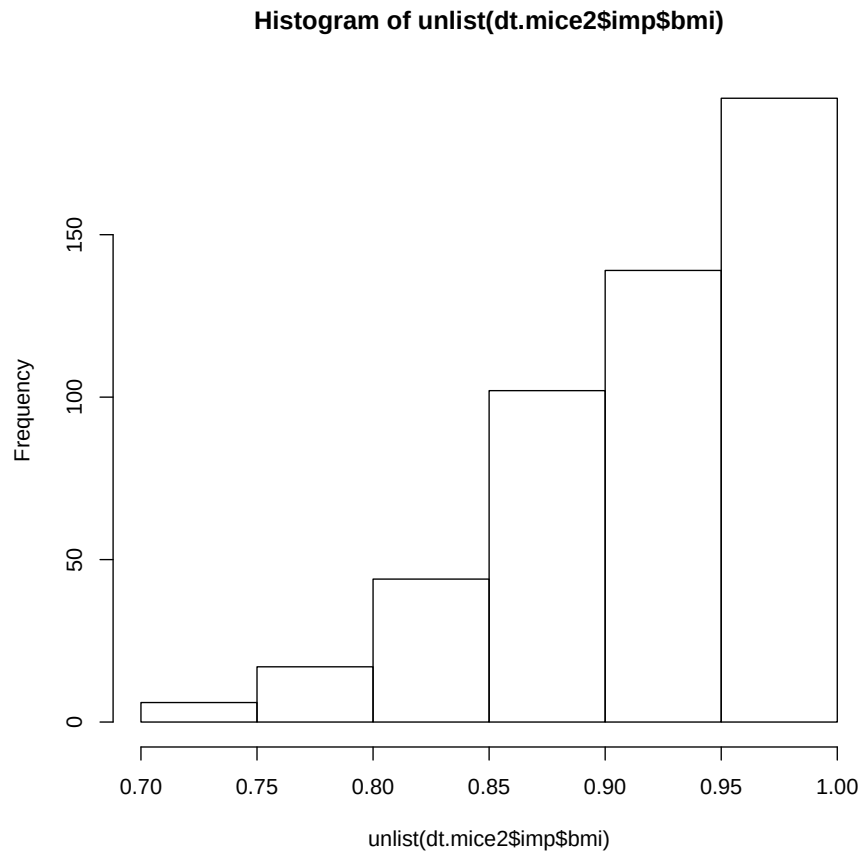
```
n.imputed <- 50 ## number of imputed datasets
set.seed(1)
dt.mice2 <- mice(dt.data,
                 m=n.imputed,
                 maxit = 1, # not relevant
                 predictorMatrix = Mlink2, # not relevant
                 method = 'SI_tnorm', # function previous define (without
                 "mice.impute.")
                 seed = 500, printFlag = FALSE)
```

Then as usual one should check that the imputed values are satisfying:

```
quantile(unlist(dt.mice2$imp$bmi))
```

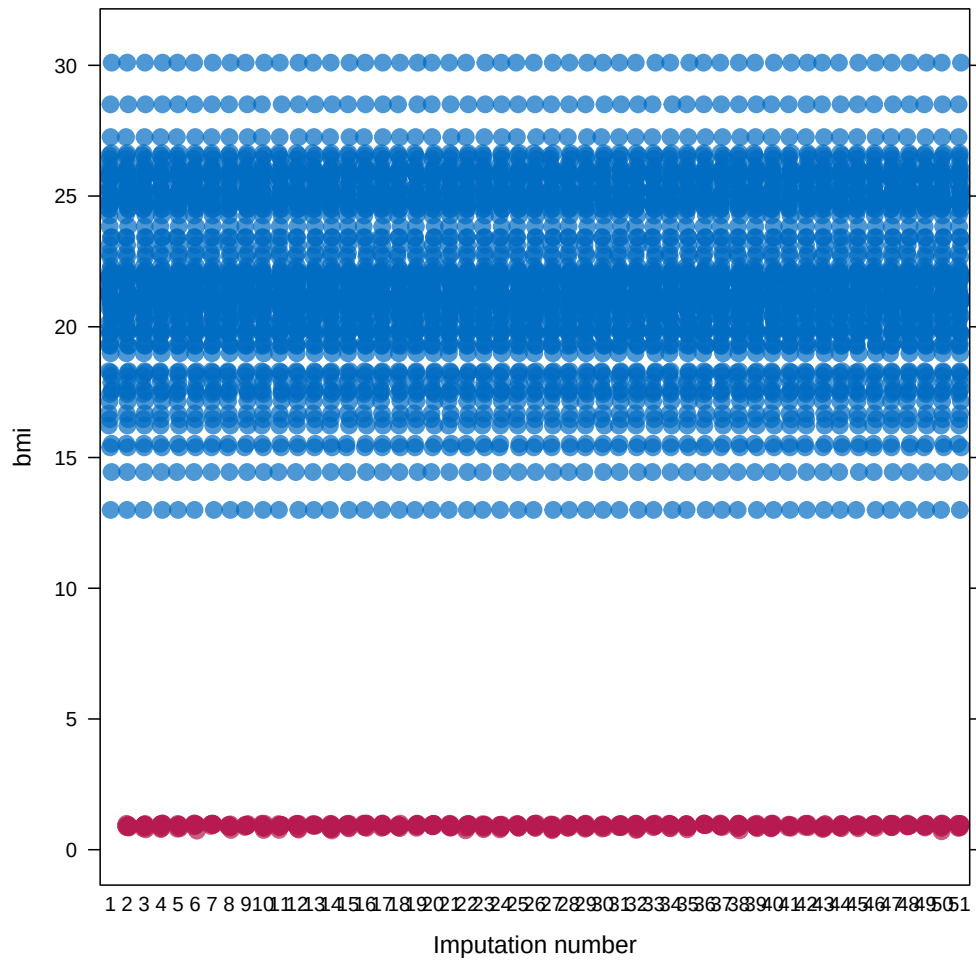
```
      0%      25%      50%      75%     100%
0.7041556 0.8790477 0.9317021 0.9687630 0.9997288
```

```
hist(unlist(dt.mice2$imp$bmi))
```



One more plot:

```
stripplot(dt.mice2, bmi~.imp, pch=20, cex=2)
```



Here for instance the imputed values does not overlap the observed one so something (i.e. the parameters of the distribution used for the imputation) is wrong.

5 Reporting guideline

From <https://stefvanbuuren.name/Winnipeg/Lectures/Winnipeg.pdf>:

- Amount of missing data
- Reasons for missingness
- Differences between complete and incomplete data
- Method used to account for missing data
- Software
- Number of imputed datasets
- Imputation model
- Derived variables
- Diagnostics
- Pooling
- Listwise deletion
- Sensitivity analysis

6 Reference

van Buuren, S. (2018). *Flexible Imputation of Missing Data*. Chapman & Hall/CRC Interdisciplinary Statistics. Chapman and Hall/CRC, Boca Raton, FL, 2 edition.